

Table of content

Data Type Size Exploration.....	2
Figure 1: Code of Data Type Size Exploration.....	2
Figure 2: Program Output.....	3
Memory Segment Address Mapping.....	3
Figure 3: Code of Memory Segment Address Mapping.....	4
Figure 4: Program Output.....	5

Data Type Size Exploration

Objective

To use the `sizeof()` operator to observe how much memory different C data types occupy on the Linux system.

Source code

```
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char:          %zu bytes\n", sizeof(char));
    printf("int:             %zu bytes\n", sizeof(int));
    printf("float:           %zu bytes\n", sizeof(float));
    printf("double:          %zu bytes\n", sizeof(double));
    printf("long:            %zu bytes\n", sizeof(long));
    printf("unsigned int:    %zu bytes\n", sizeof(unsigned int));
    printf("long long:       %zu bytes\n", sizeof(long long));
    printf("pointer:         %zu bytes\n", sizeof(int*));

    return 0;
}
```

Figure 1: Code of Data Type Size Exploration

Compilation and output

```
vboxuser@ubuntu:~/lab4$ nano datatype_sizes.c
vboxuser@ubuntu:~/lab4$ gcc -Wall datatype_sizes.c -o datatype_sizes
vboxuser@ubuntu:~/lab4$ ./datatype_sizes
=== Data Type Sizes on This System ===

char:      1 bytes
int:       4 bytes
float:     4 bytes
double:    8 bytes
long:      8 bytes
unsigned int: 4 bytes
long long: 8 bytes
pointer:   8 bytes
vboxuser@ubuntu:~/lab4$ S
```

Figure 2: Program Output

Observation

The `sizeof()` operator reveals the exact byte allocations for standard C data types on the execution environment. On a standard 64-bit Linux architecture, types like `long` and `pointers` expand to 8 bytes to align with the system's word size, whereas they occupy only 4 bytes on 32-bit systems.

Summary

This exercise highlights how data type sizes serve as a contract between the programmer, compiler, and OS. Using `sizeof()` confirms system-dependent memory allocation rules and illustrates how architecture impacts variable storage requirements.

Memory Segment Address Mapping

Objective

To observe the memory addresses of variables stored in different process memory segments, including Data, BSS, Stack, and Heap

Source code

```
GNU nano 8.7.1 memory_segments.c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

// Data Segment: Initialized global variable
int global_var = 150;

// BSS Segment: Uninitialized global variable
int bss_var;

int main() {
    // Stack Segment: Local variable
    int local_var = 30;

    // Heap Segment: Dynamically allocated memory
    int *heap_var = (int*)malloc(sizeof(int));
    if (heap_var == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        return 1;
    }
    *heap_var = 500;

    printf("..... Variable Addresses ..... \n\n");
    printf("global_var (Data):      %p\n", (void*)&global_var);
    printf("bss_var (BSS):             %p\n", (void*)&bss_var);
    printf("heap_var (Heap Data):       %p\n", (void*)heap_var);
    printf("heap_var (Stack Ptr):      %p\n", (void*)&heap_var);
    printf("local_var (Stack):         %p\n", (void*)&local_var);

    // Address Difference calculation
    uintptr_t stack_addr = (uintptr_t)&local_var;
    uintptr_t heap_addr = (uintptr_t)heap_var;

    printf("\nAddress Difference (Stack - Heap): %lu bytes\n",
        (unsigned long)(stack_addr > heap_addr ? stack_addr - heap_addr : heap_addr - stack_addr));

    free(heap_var);
    return 0;
}
```

Figure 3: Code of Memory Segment Address Mapping

Compilation and output

```
vboxuser@ubuntu:~/lab4$ nano memory_segments.c
vboxuser@ubuntu:~/lab4$ gcc -Wall memory_segments.c -o memory_segments
vboxuser@ubuntu:~/lab4$ ./memory_segments

..... Variable Addresses .....

global_var (Data):      0x5f407e5a2010
bss_var (BSS):          0x5f407e5a202c
heap_var (Heap Data):   0x5f4085f0a010
heap_var (Stack Ptr):   0x7ffff65c4c50
local_var (Stack):      0x7ffff65c4c4c

Address Difference (Stack - Heap): 36006596947004 bytes
vboxuser@ubuntu:~/lab4$ S
```

Figure 4: Program Output

Memory Segment Identification

global_var	Data
bss_var	BSS
heap_var	Heap Data
heap_var	Stack Ptr
Local_var	Stack

Observation

Printing variable pointers confirmed that local stack variables reside at high virtual memory addresses (0x7ffd...), whereas global, static, and heap variables reside at lower addresses (0x55...). Execution across different runs may shift exact addresses due to

Address Space Layout Randomization (ASLR), but the relative segment hierarchy remains unchanged.

Summary

This exercise demonstrated the structural division of process memory into the Stack, Heap, Data, and BSS segments. It illustrated how the OS prevents memory collisions by growing the Stack downward and the Heap upward, while using stack pointers to reference dynamically allocated heap memory.